FixTrace

AI-Assisted Maintenance & Asset Lifecycle Management SaaS

A multi-tenant platform where every physical asset gets a digital identity (QR code), every fault becomes a trackable work order, and every repair builds a permanent maintenance history.

What this document covers
1. What FixTrace is
2. The problem it solves
3. How FixTrace solves it
4. Users and roles
5. Core modules
6. Maintenance request lifecycle
7. Technology stack and why
8. Architecture overview
9. Engineering practices
10. Development roadmap

1. What is FixTrace?

FixTrace is a **multi-tenant SaaS** for maintenance management and asset lifecycle tracking. Many organisations can use the same platform, while each organisation's data stays completely isolated.

Who uses it

- Colleges and universities
- Hostels and residential societies
- Apartments and property management companies
- Offices and corporate facilities
- Hospitals and other organisations
- Hotels and commercial buildings

What they manage

Physical assets such as air conditioners, elevators, generators, computers, laboratory equipment, electrical systems, water pumps, furniture and networking equipment.

Core concept

Every asset gets a unique digital identity with a QR code. A user scans it to report a fault; the maintenance team receives, assigns, tracks, resolves and verifies the request. Over time, FixTrace builds a complete maintenance history for every asset. The long-term vision is to move maintenance from **reactive repair** to **data-driven asset lifecycle management**.

2. The Problem

Most organisations today manage maintenance through phone calls, WhatsApp messages, spreadsheets, registers or informal communication. This causes:

Problem	Real-world consequence
Requests get lost	A broken AC is reported verbally and never reaches anyone.
No clear ownership	Nobody knows who is responsible for a repair.
No progress visibility	Reporters keep asking "has it been fixed yet?"
No maintenance history	Past repairs, parts and costs are unknown.
Repeated failures	The same equipment keeps failing; the pattern is never noticed.
No technician accountability	No reliable record of technician performance.
Missed deadlines	No SLA tracking or escalation.
Poor cost insight	Managers cannot analyse costs or recurring failures.

3. How FixTrace Solves It

Problem	FixTrace solution
Lost requests	QR-based fault reporting creates a structured work order instantly.
Unclear ownership	Managers assign technicians; every work order has one accountable owner.
No visibility	Status workflow with notifications and a full activity timeline.
No history	Digital Asset Passport: a chronological record of faults, repairs, parts, photos, downtime and cost.
Recurring faults	Analytics and recurring-failure detection highlight problem assets.
No accountability	Technician profiles, workload and resolution-time statistics.
Missed deadlines	Configurable SLA engine, background monitoring and escalation rules.
Cost blindness	Dashboards built from real database data (costs, trends, SLA compliance).

4. Users and Roles (RBAC)

Permissions are enforced on the **backend**, not just hidden in the UI.

Role	Main responsibilities
Platform Super Admin	Manage the platform, view/suspend/activate organisations, monitor system health and usage.
Organisation Admin	Organisation settings, invite members, departments and locations, register assets, assign roles, view all requests, analytics.
Facility Manager	Review incoming requests, assign technicians, set priority and deadlines, monitor SLA, escalate delays.
Technician	View assigned work orders, accept/reject, update progress, add notes, upload before/after photos, record parts used, mark complete.
Reporter / General User	Scan QR codes, report faults with photos, track own requests, confirm whether the issue was resolved.

5. Core Modules

#	Module	What it does
1	Authentication & Organisation Management	Register/login, password hashing, JWT access + refresh tokens, session revocation, password reset, invitations, memberships, RBAC, multi-tenant isolation. Designed so one user can later belong to multiple organisations.
2	Asset Management	Asset details (code, category, manufacturer, model, serial, purchase/installation/warranty dates, location, department, condition, status, photos, metadata) and a unique QR code per asset.
3	QR-Based Fault Reporting	Scan QR, describe fault, choose category and severity, upload photos, system creates a request and notifies the facility manager. QR URLs must not leak confidential data.
4	Maintenance Request & Work Orders	Full workflow with valid state transitions, activity records for every important change, and safe handling of concurrent updates.
5	Technician Management	Profiles, skills, availability, workload, completed work, average resolution time. Smart assignment comes later.
6	Maintenance History (Digital Asset Passport)	Permanent chronological history: faults, assignments, repairs, parts, photos, inspections, downtime, costs.
7	Notifications & SLA	In-app notifications (email later), configurable response/resolution targets per priority, overdue detection, escalation, SLA compliance history, background jobs.
8	Analytics & Dashboard	Asset counts, open/in-progress/overdue requests, resolution time, technician workload, most-failing assets, costs, SLA compliance, monthly trends.
9	Intelligent Insights (future)	Recurring-failure detection, fault summaries, cause suggestions, preventive maintenance recommendations. AI output is clearly labelled and never acts without human approval.

6. Maintenance Request Lifecycle

Statuses: **OPEN, UNDER_REVIEW, ASSIGNED, ACCEPTED, IN_PROGRESS, ON_HOLD, RESOLVED, VERIFIED, CLOSED, REJECTED**. Only valid transitions are allowed, and each important change creates an activity record.

Typical happy path

OPEN → UNDER_REVIEW → ASSIGNED → ACCEPTED → IN_PROGRESS → RESOLVED → VERIFIED → CLOSED

Side paths

- **ON_HOLD**: work paused (e.g. waiting for spare parts), then back to IN_PROGRESS.
- **REJECTED**: request invalid or duplicate (reviewer decision).
- A technician rejecting an assignment returns the request to review for re-assignment.
- If the reporter says the issue is not fixed, the request is reopened for further work.

Each work order stores

Unique ID, related asset, reporter, assigned technician, priority, description, fault category, created/assigned/expected/resolved timestamps, repair notes, evidence, current status and complete activity history.

Concurrency

Two managers must not silently overwrite each other's changes. This is handled with controlled updates (e.g. optimistic locking with a version field) and database transactions.

7. Technology Stack

Layer	Technology	Why
Frontend	React, TypeScript, Vite, Tailwind CSS, React Router	Modern, fast, type-safe UI with reusable components.
Data and forms	TanStack Query, React Hook Form, Zod	Server-state caching, performant forms, shared validation.
Charts	Recharts (or equivalent)	Dashboards and trend visualisation.
Backend	Node.js, TypeScript, Express.js or NestJS (to be recommended)	One language across the stack; REST API first.
Validation / Auth	Zod, JWT (access + refresh)	Strict request validation; stateless authentication with revocable sessions.
Database	PostgreSQL + Prisma ORM	Strong relational modelling, constraints, indexes, transactions, migrations.
Caching / Jobs	Redis, BullMQ	Only where justified: SLA monitoring, notifications, async work.
File storage	S3-compatible object storage	Fault photos, asset images, repair evidence.

Layer	Technology	Why
DevOps	Docker, Docker Compose, Git/GitHub, GitHub Actions	Reproducible environments and CI/CD.
Testing	Jest or Vitest	Unit, integration and end-to-end tests.

Design choice: a **modular monolith** (not microservices) with clear module boundaries, mostly free/open-source tools, and no technology added just for show.

8. Architecture Overview

High-level flow of the system:

Layer	Components
Clients	React web app (desktop and mobile browser); QR scan opens the asset page / report form
API layer	REST API: routing, authentication, RBAC, validation, rate limiting, error handling, request IDs
Business modules	Auth/Org, Assets, Fault Reporting, Work Orders, Technicians, Notifications/SLA, Analytics (controllers separated from services)
Data layer	PostgreSQL via Prisma (transactions, constraints, indexes)
Async layer	BullMQ workers backed by Redis: SLA checks, escalations, notifications
Storage	S3-compatible bucket for images and evidence

Multi-tenancy

All organisation-owned tables carry an **organisation_id**. Every query is scoped by the organisation of the authenticated membership, and tests verify that one tenant can never read another tenant's data. Because users connect to organisations through a **membership** (user + organisation + role), a user can belong to several organisations in future without redesign.

Key database entities (proposed)

Users, Organisations, Memberships, Roles/Permissions, Invitations, Sessions/Refresh tokens, Departments, Locations, Asset Categories, Assets, Asset Photos, Maintenance Requests/Work Orders, Activity Logs, Attachments, Technician Profiles and Skills, Parts Used, Notifications, SLA Policies, SLA Records, Audit Logs. The detailed ER diagram is part of Phase 0.

9. Engineering Practices

Area	Approach
Security	Strong password hashing, backend RBAC, tenant isolation, input validation, SQL-injection protection via Prisma, rate limiting, secure file uploads, secrets in environment variables.
Reliability	Structured logging, centralised error handling, correlation IDs, retries only where safe, idempotent background jobs.
Testing	Auth, authorisation, tenant isolation, asset creation, QR reporting, assignment, status transitions, SLA calculations, notifications, critical transactions.
Documentation	README, ARCHITECTURE, DATABASE, API, DEVELOPMENT, CHANGELOG, TASKS and a project-status file for handoff.
Honesty	No fabricated users or metrics; history is not called tamper-proof unless a real tamper-evident mechanism exists; AI suggestions are separated from verified facts.

10. Development Roadmap

Phase	Focus	Key deliverables
0	Product and architecture design	Requirements, edge cases, architecture, ER diagram, API structure, security risks, roadmap, folder structure. No coding.
1	Core foundation	Project setup, DB and Prisma, authentication, organisations, memberships, RBAC, tenant isolation.
2	Asset management	Asset CRUD, categories, locations/departments, QR generation, detail pages, search and filters.
3	Maintenance workflow	Fault reporting, work orders, assignment, status transitions, repair updates, verification, activity history.
4	Operational intelligence	Notifications, Redis, background jobs, SLA monitoring, escalation, audit logs, dashboards.
5	Advanced features	Digital Asset Passport, smart assignment, recurring failures, AI fault summaries, preventive recommendations.
6	Production readiness	Test coverage, performance, security review, Docker deployment, CI/CD, monitoring, API and user docs.

Why this is a strong SDE portfolio project

- Full-stack development with strong relational database design
- Multi-tenant SaaS architecture and secure authentication/authorisation
- Complex business workflows and concurrency handling
- Reliable background processing and thoughtful API design
- Testing, debugging, deployment and production readiness
- Meaningful (not decorative) use of AI